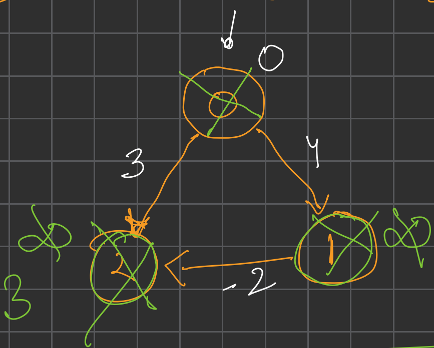
Bellmon Ford (Code easy) ✓

- → Directed graph ✓ (Mandatory)
- → Negative weight apply (~~✗~~)

negative Cycle ✗

$\boxed{\text{Dijkstra Algo}}$ ✓



src 0

✓ $0 \xrightarrow{} 0$

$0 \xrightarrow{4} 1$

$0 \xrightarrow{2} 2$

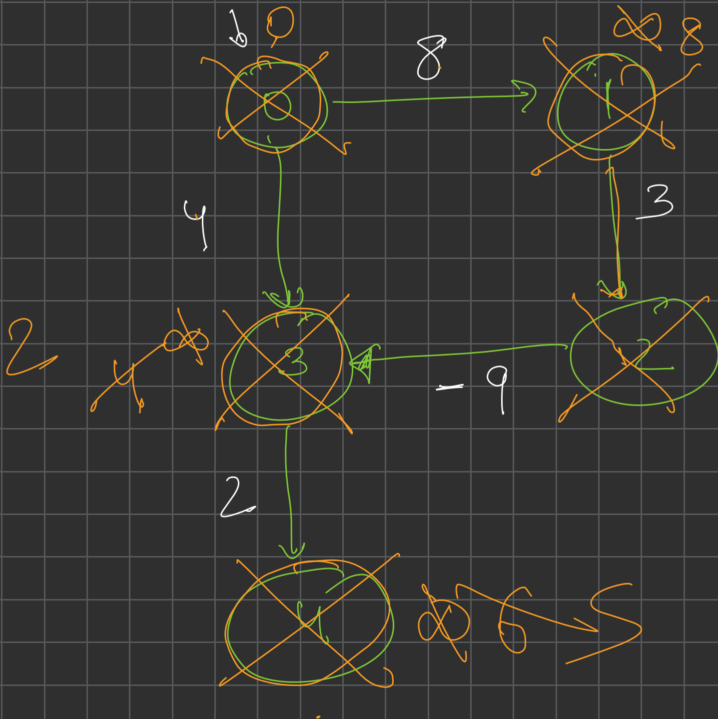
$0 \xrightarrow{0} 0$ $\quad$ $0 \xrightarrow{3} 2$ $\quad$ $0 \xrightarrow{4} 1$

dijkstra algo ⇒ $\boxed{\text{visited node}}$ ✓

$\boxed{-ve}$

→ Distance

→ Time



5 node   [1 last step] -ve

4 time → 1 more time → Negative cycle

more

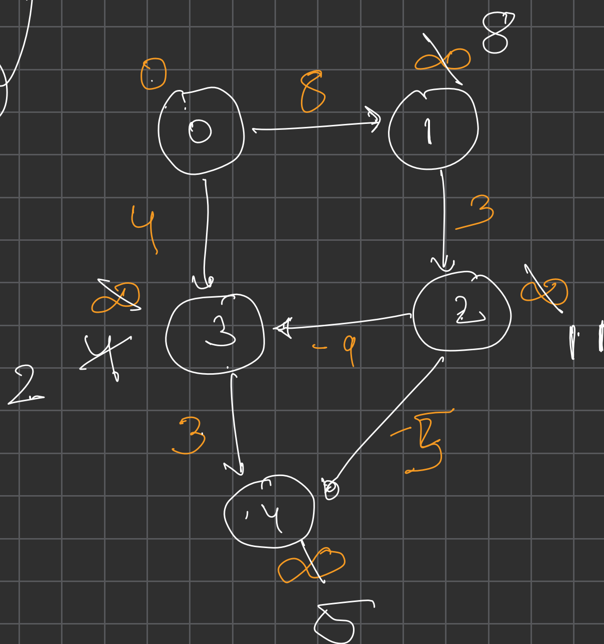
1 cycle

2 cycle

3 1

Bellmon ford

Relax all the edges



1 → 2 (3)

2 → 3 (-9)

3 → 4 (3)

0 → 1 (8)

2 → 4 (-5)

0 → 3 (4)

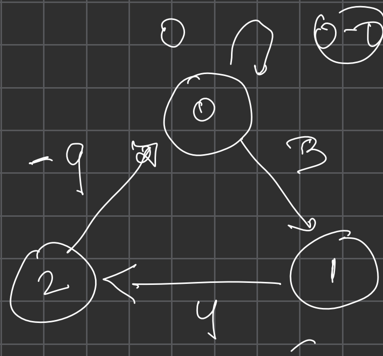
→ Path exist

$\infty - 5 < \infty$ ⇒ hoga

N node : (N-1) time Relax edges

1 time : Relax edges. if distance change

-ve cycle ×



Shortest

× answer

0 1 2 0 → -2

0 1 2 0 1 2 0 → -4

0 1 2 0 1 2 0 1 2 0 → -6

N-1 + Relax ?

$E * (V-1) + E \to EV$

$V \to V-1$

$4 \to$ 3 time

2 time

3 time

3 time

1 more time

repeat

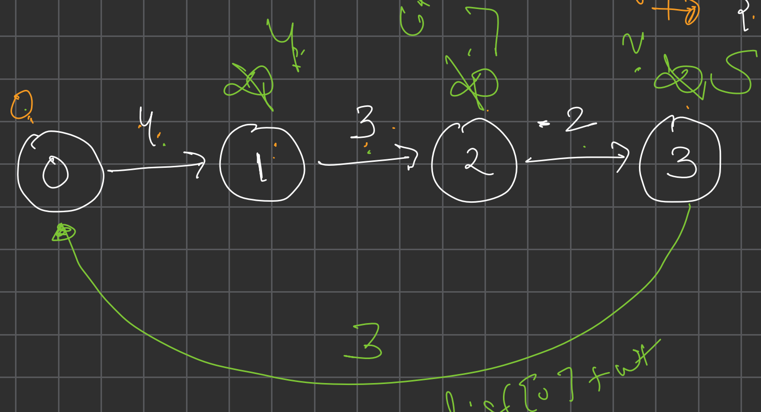
1 ite complete

{3, 0} -1

{2, 3} -2

{1, 2} → 3

{0, 1} 4

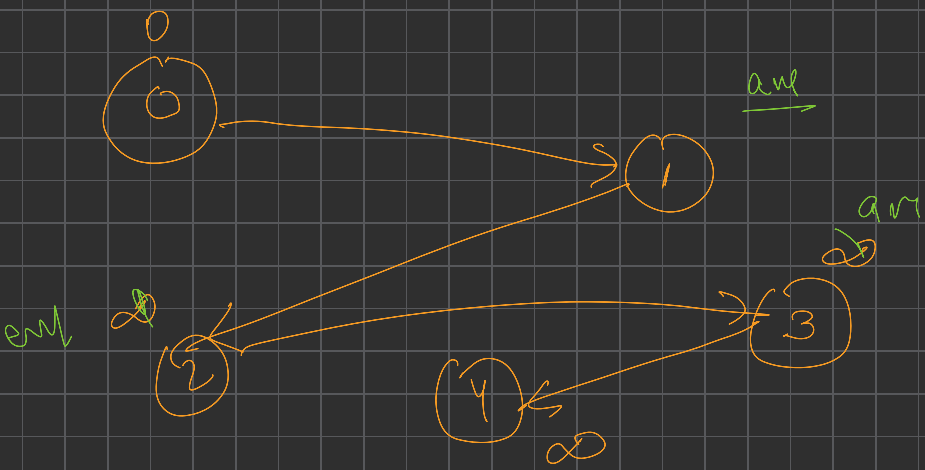


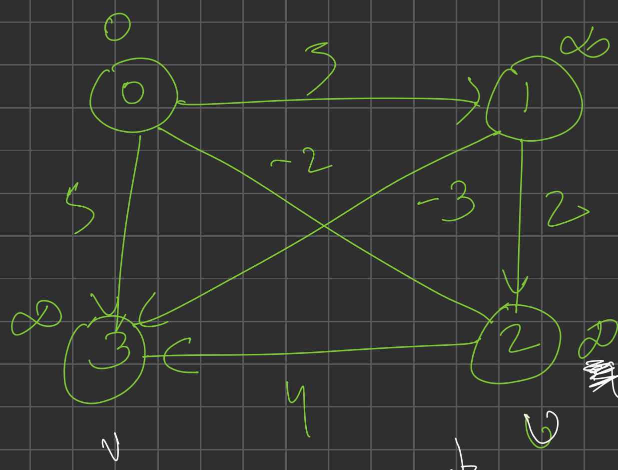
+

1 step

dist[v] > dist[u] + wt

5 > 7 - 2





108

INT

$dist[v] > dist[u] + wt$

$\infty > \infty + 4$

INT_Max + 4

integer overflow

Undirected graph $\nexists$ Bellmond ford

$\hookrightarrow$ -ve weight

0

0 - 0

0 - 0

0 -1- 0

-4

$0 \xrightarrow{-2} 1$

Directed graph

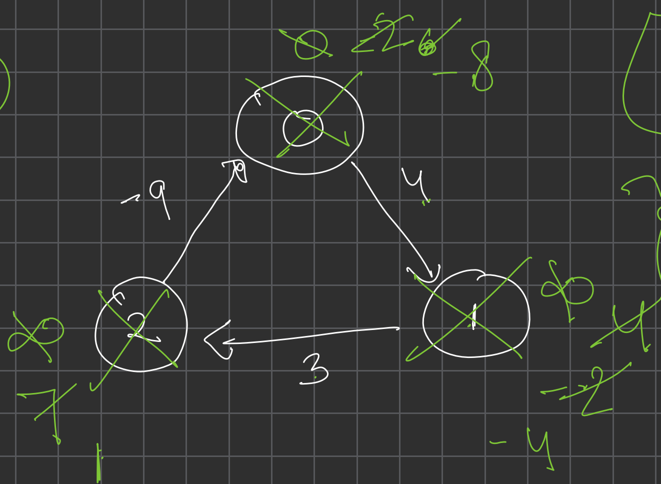
$\nexists$ -ve cycle ✓

Directed graph (-ve weight)

Dijkstra Algorithm

ensure nikol jayega

-2



2

Ruko

Stop kab hona hai